Lab Record: Networking Programs and Packet Sniffer

Output screenshots with step-by-step explanation

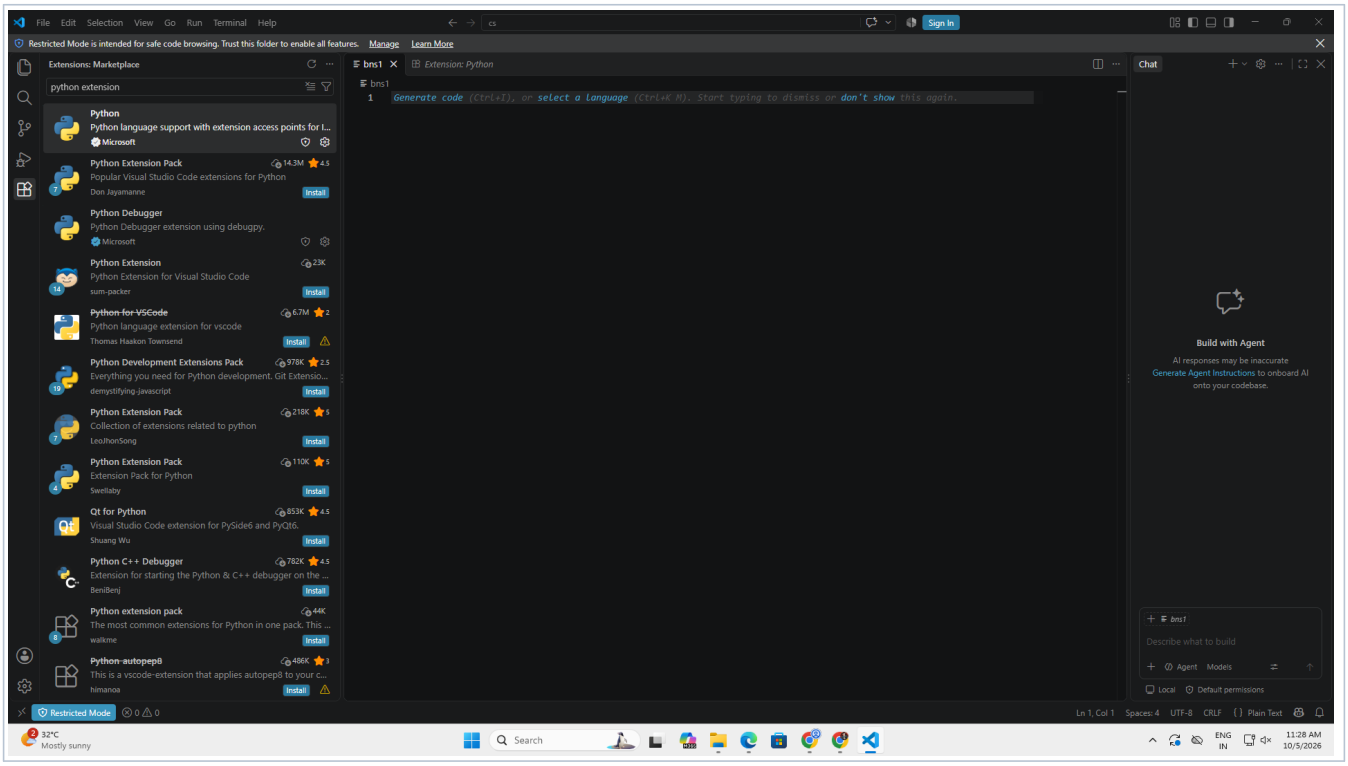
Contents

Part A: Basic network sniffer in Python (VS Code setup, installation, running)
Part B: ARP and RARP address resolution program (Java)
Part C: Distance vector routing, shortest distance matrix (Java)

Part A: Basic network sniffer in Python

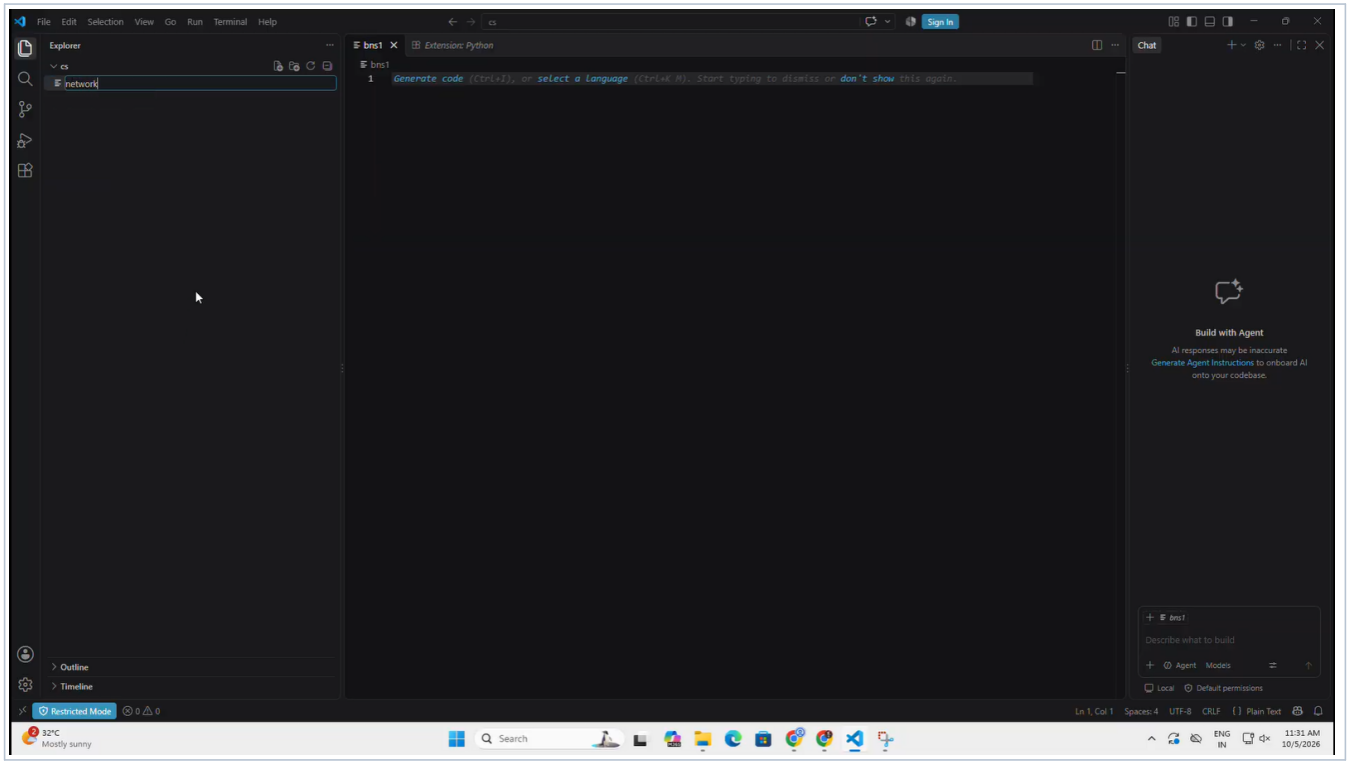
Aim: To write a basic network sniffer in Python that captures packets on the local machine and shows source and destination IP, protocol (TCP/UDP/ICMP) and a payload preview. **Tools:** Visual Studio Code, Python 3.14, Scapy library, Npcap (packet capture driver on Windows). Use only on your own machine or a network you are authorised to monitor.

Step 1: Install the Python extension
Open the Extensions view in VS Code and search for 'python extension'. Install the Microsoft Python extension, which provides editing, IntelliSense and run support.



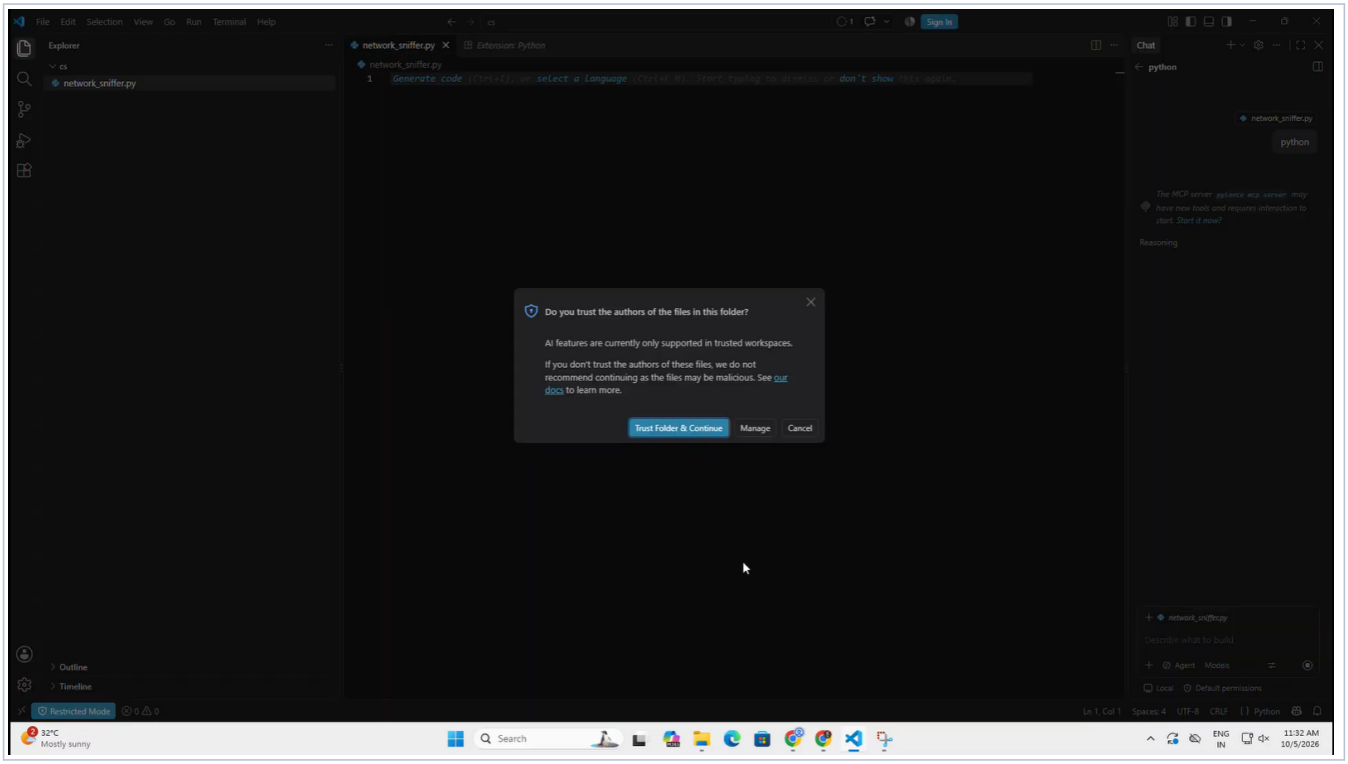
Step 2: Create the project file

In the Explorer panel, open the folder (cs) and create a new file. Name it network_sniffer.py so VS Code recognises it as Python.



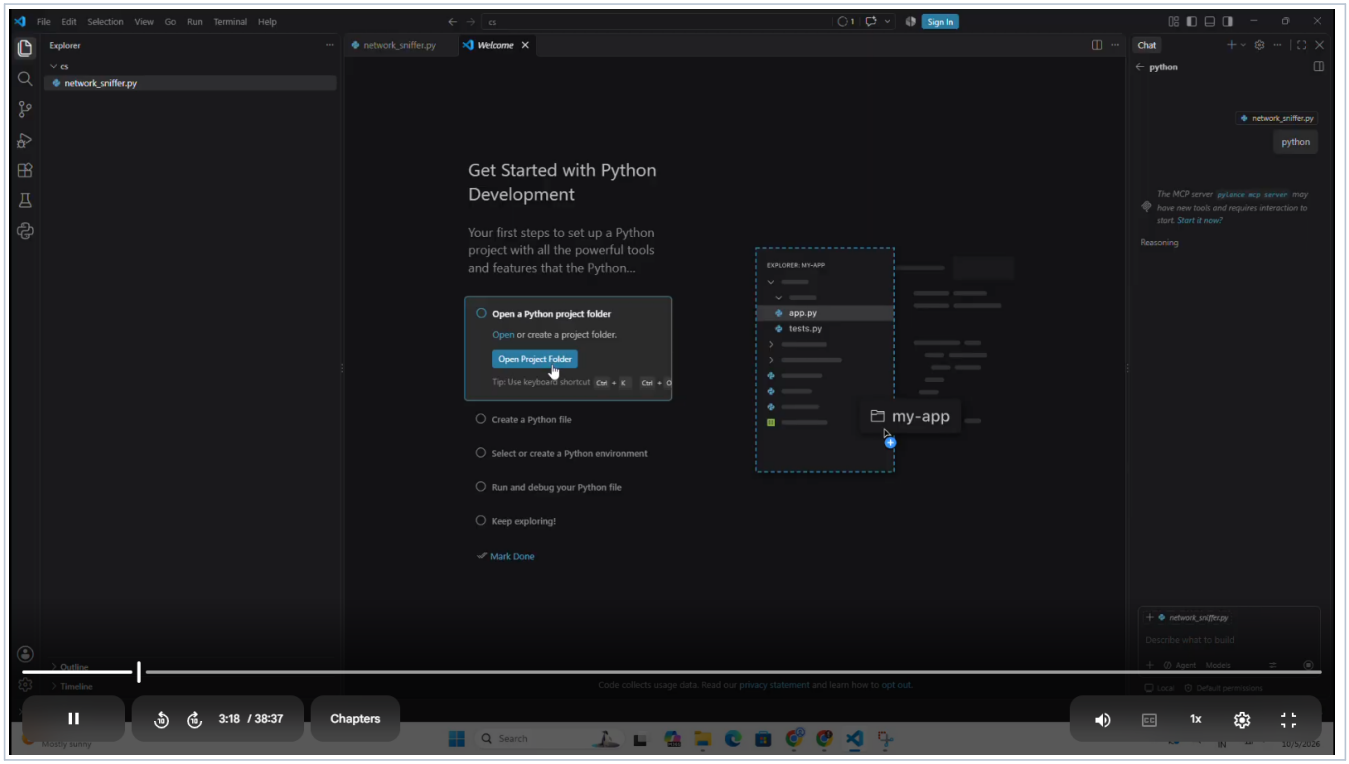
Step 3: Trust the folder

VS Code asks whether you trust the authors of the folder. Choose 'Trust Folder & Continue' so all features, including the Python tools, work.



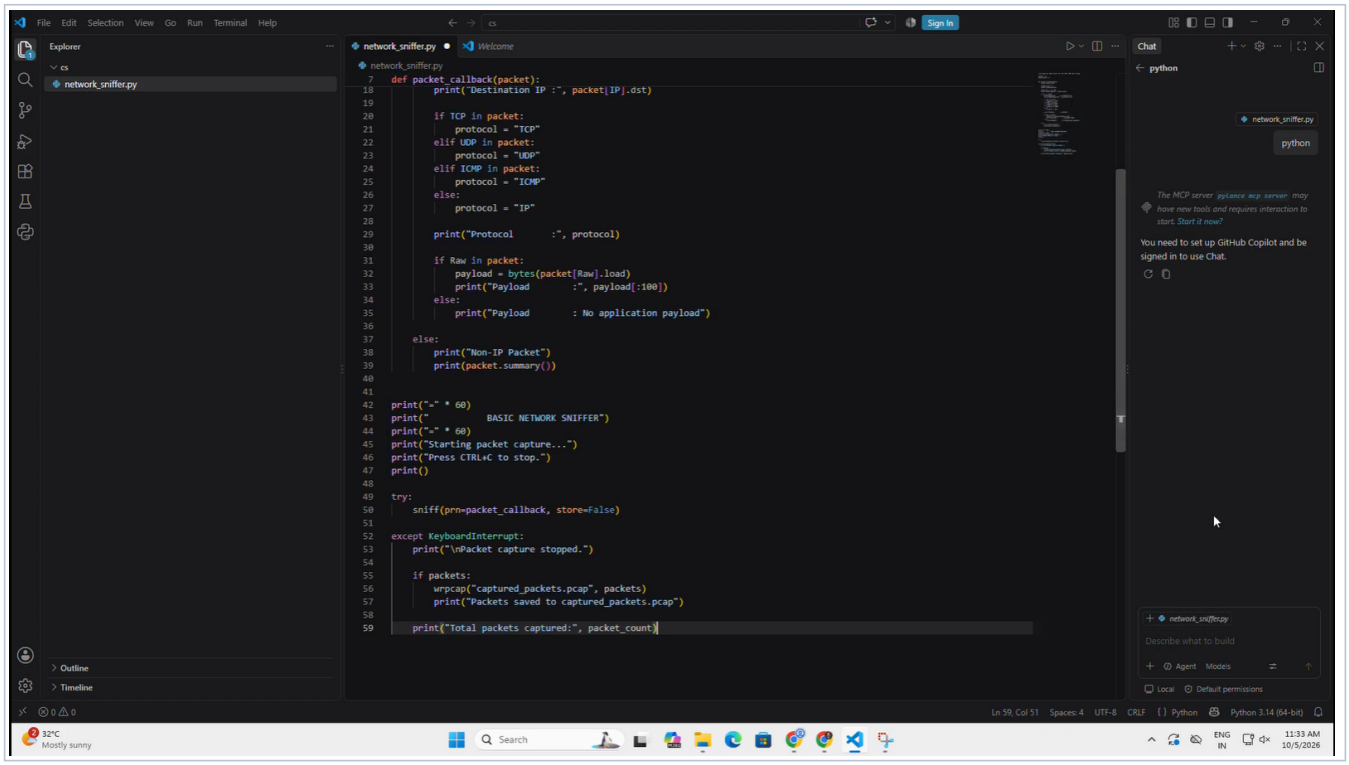
Step 4: Python environment ready

The 'Get Started with Python Development' page appears after the extension loads. The file is now treated as a Python file.



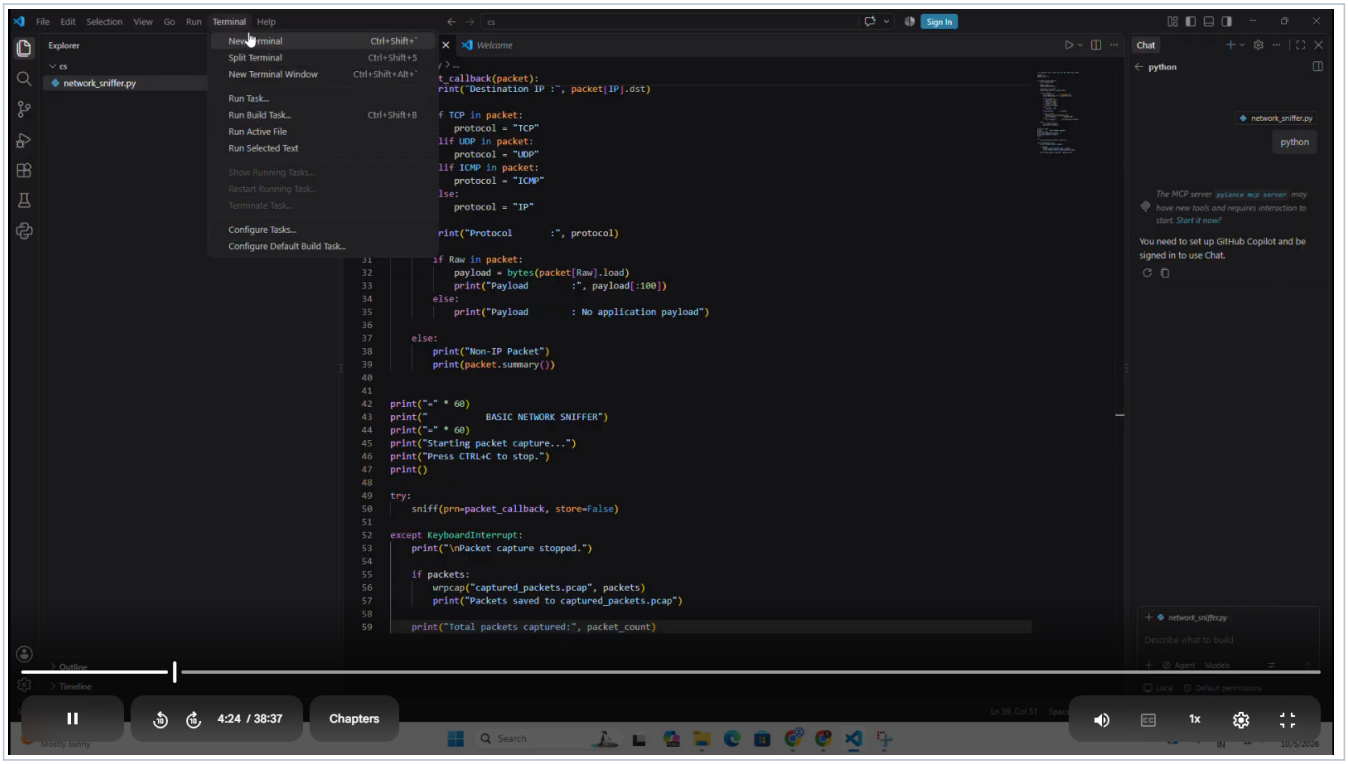
Step 5: Write the sniffer code

The script defines a packet_callback function that prints source and destination IP, detects TCP/UDP/ICMP, shows the first 100 bytes of payload, and handles non-IP packets. sniff(prn=packet_callback, store=False) starts the capture, and CTRL+C stops it and saves packets to a .pcap file.



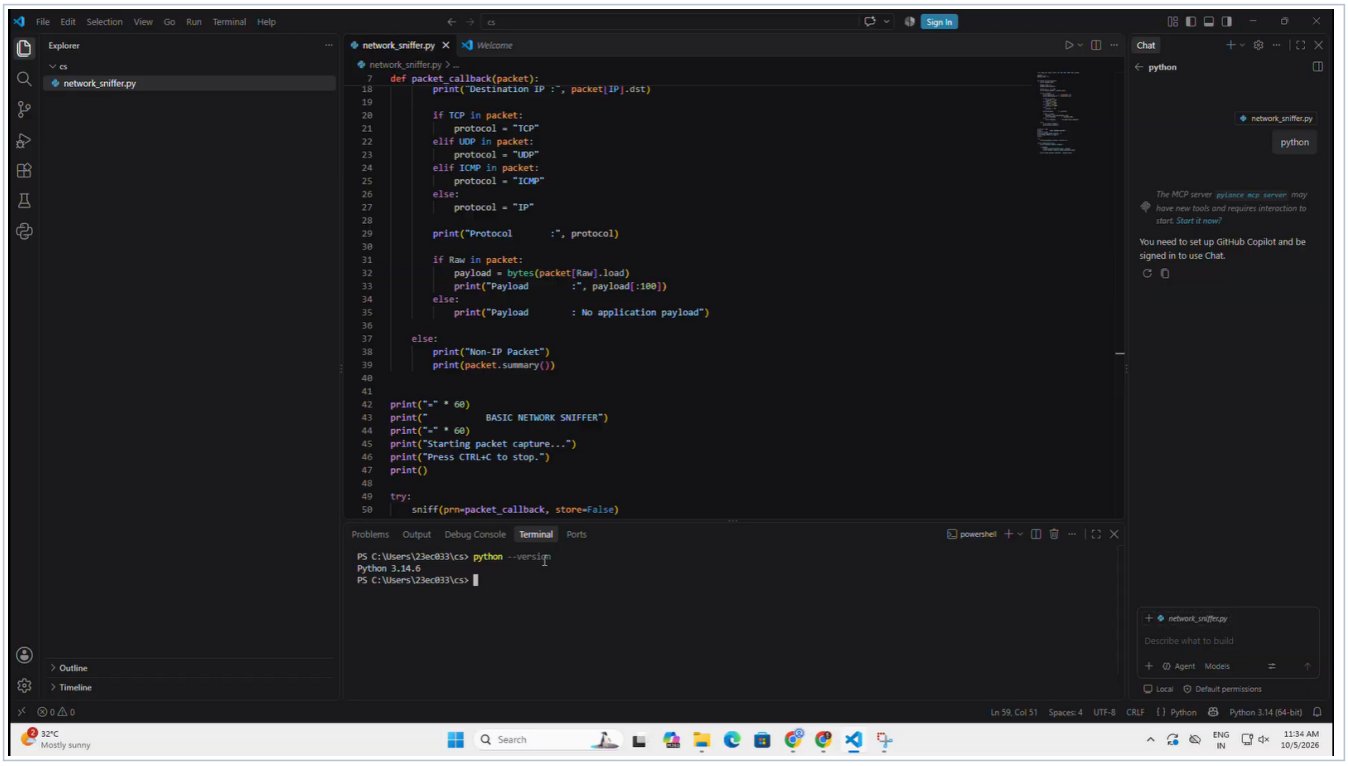
Step 6: Open the terminal

Use Terminal > New Terminal from the menu bar to open an integrated PowerShell terminal in the project folder.



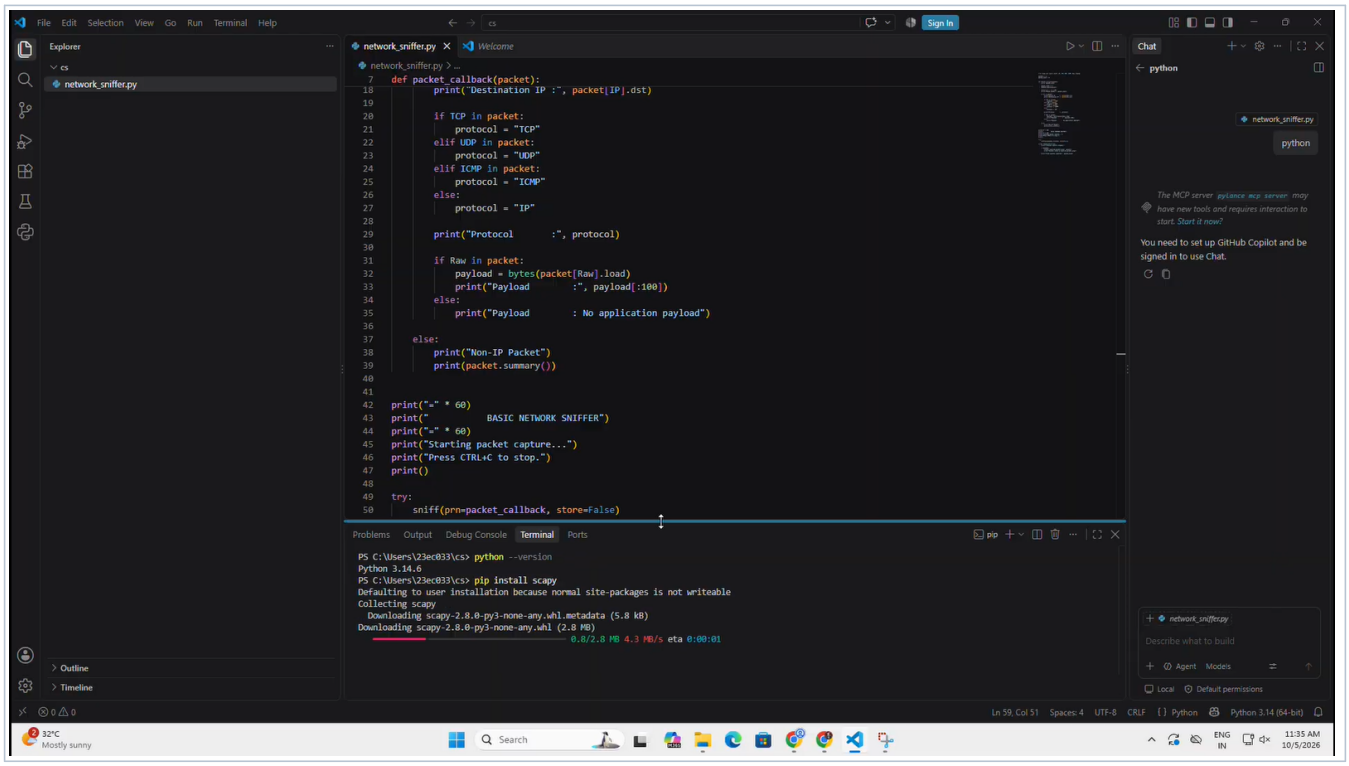
Step 7: Check the Python version

Run `python --version` to confirm Python is installed. Output: Python 3.14.6.



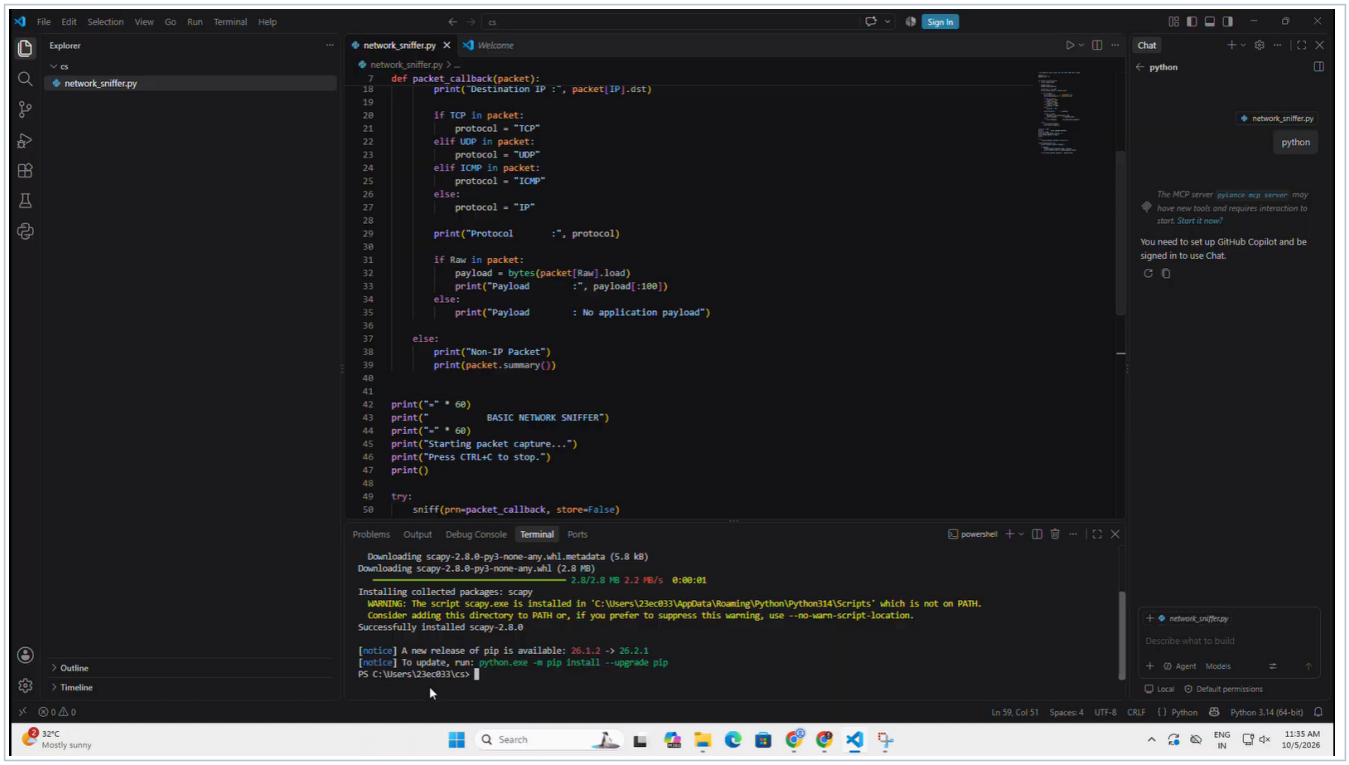
Step 8: Install Scapy

Run `pip install scapy`. Pip downloads the Scapy 2.8.0 package (about 2.8 MB).



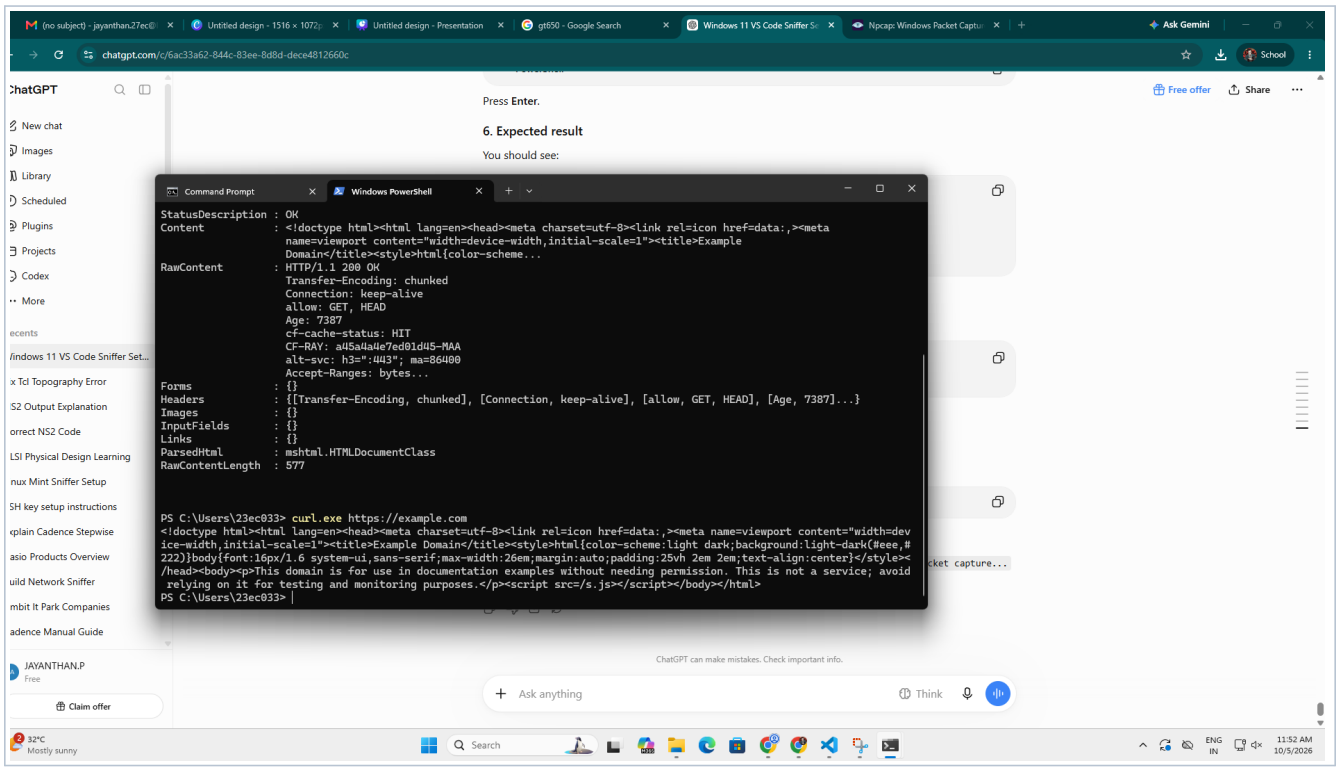
Step 9: Installation complete

Pip reports 'Successfully installed scapy-2.8.0'. The warning about the Scripts folder not being on PATH is harmless for this program; the pip upgrade notice is optional.



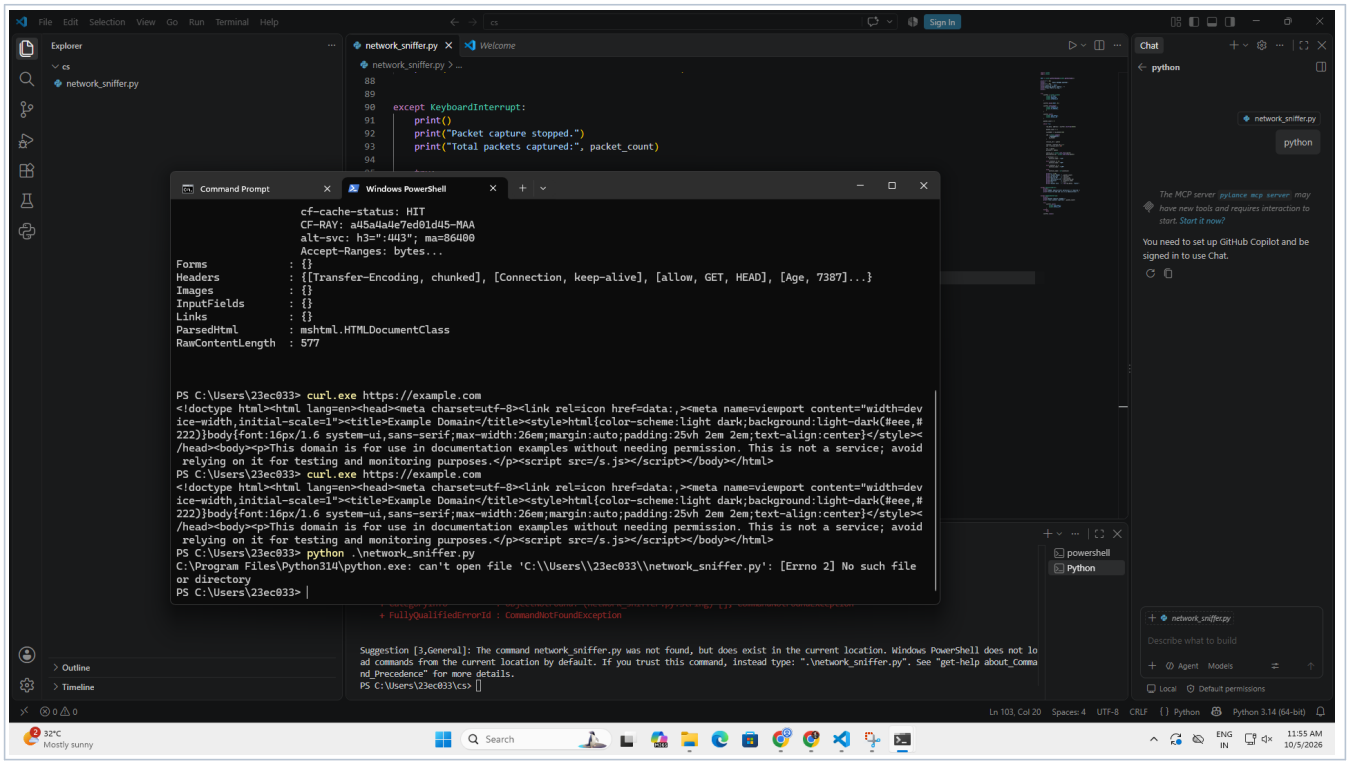
Step 10: Test internet access from PowerShell

Running `curl.exe https://example.com` returns the Example Domain page with status 200 OK, confirming the network connection works before sniffing.



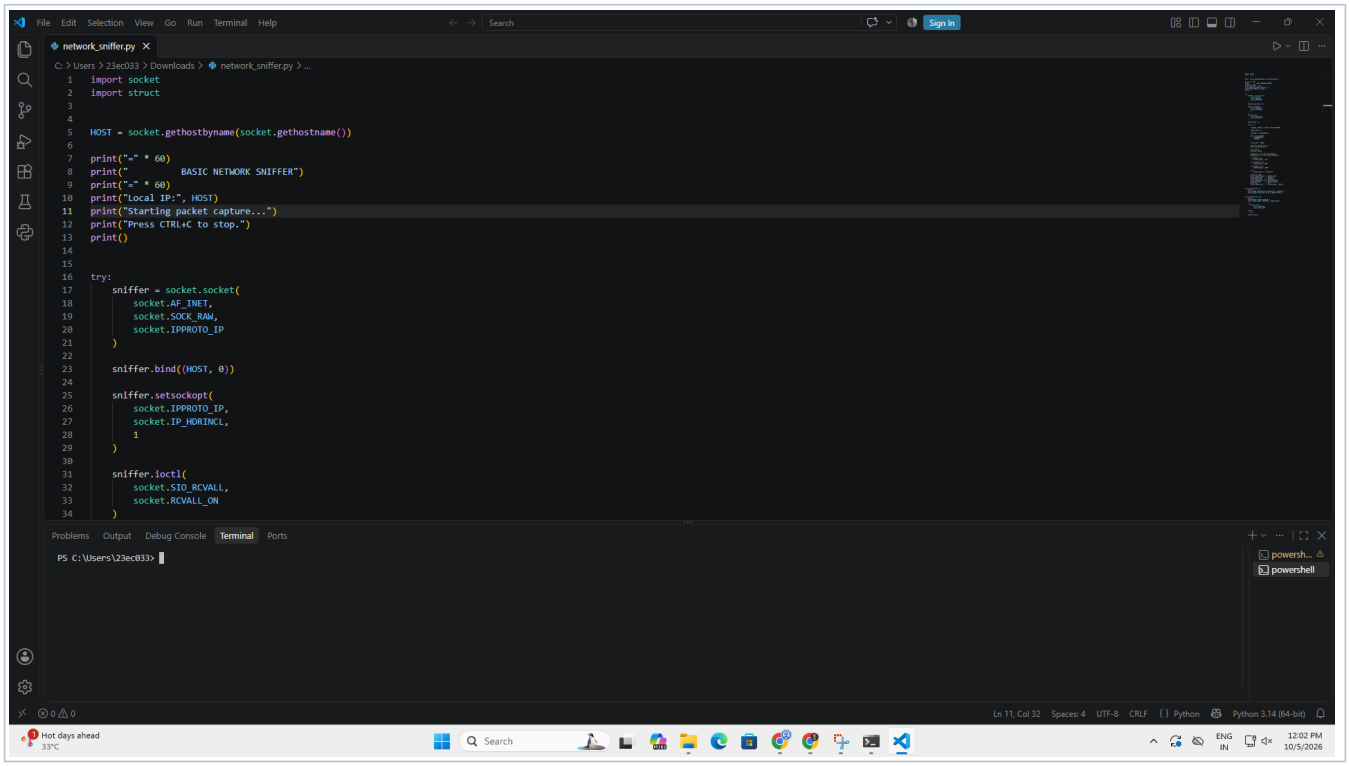
Step 11: Run from the correct folder

Running `python .\network_sniffer.py` from C:\Users\23ec033 fails with 'No such file or directory' because the file is saved elsewhere. Change into the folder that contains the file (here, Downloads) before running.



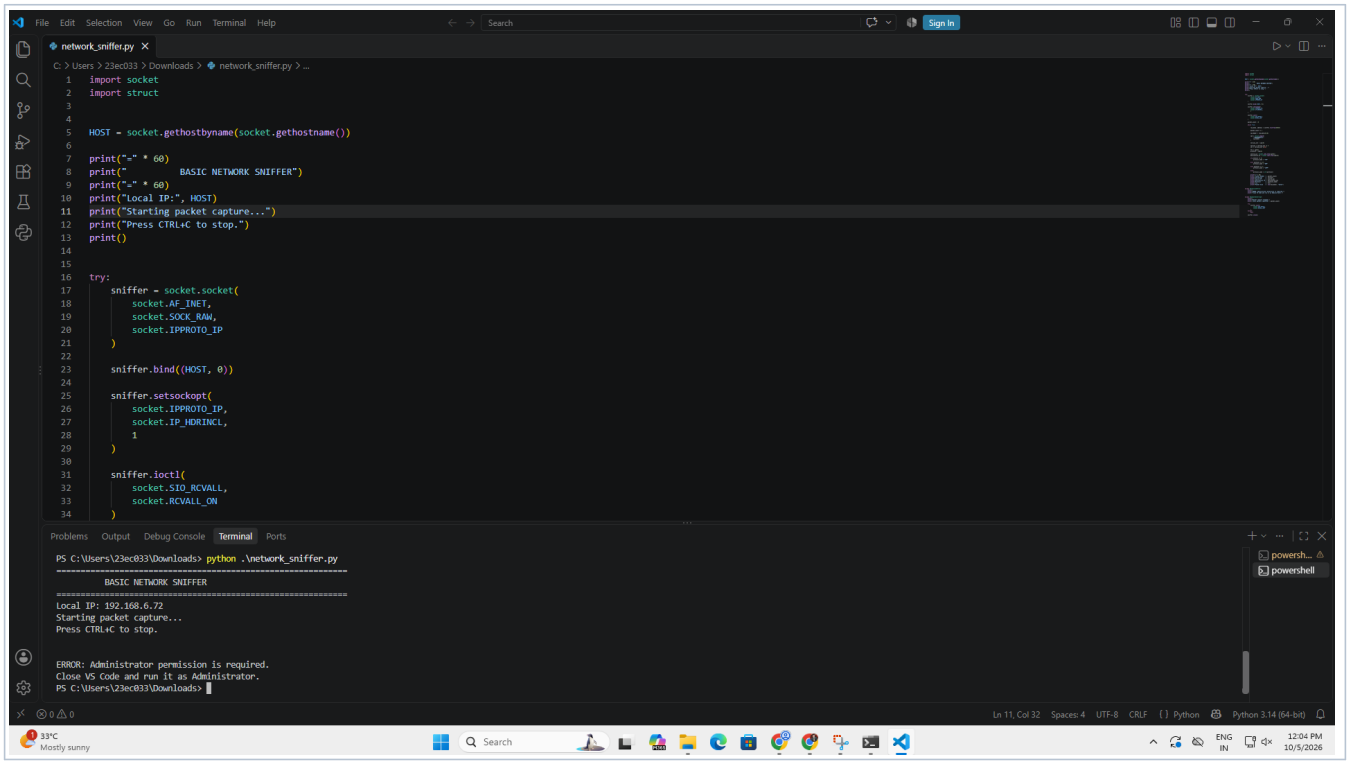
Step 12: Raw-socket version of the sniffer

A second version uses Python's socket module: it finds the local IP, creates a raw socket (AF_INET, SOCK_RAW, IPPROTO_IP), binds to the host, includes IP headers (IP_HDRINCL) and enables promiscuous receive with SIO_RCVALL.



Step 13: Run result: administrator permission needed

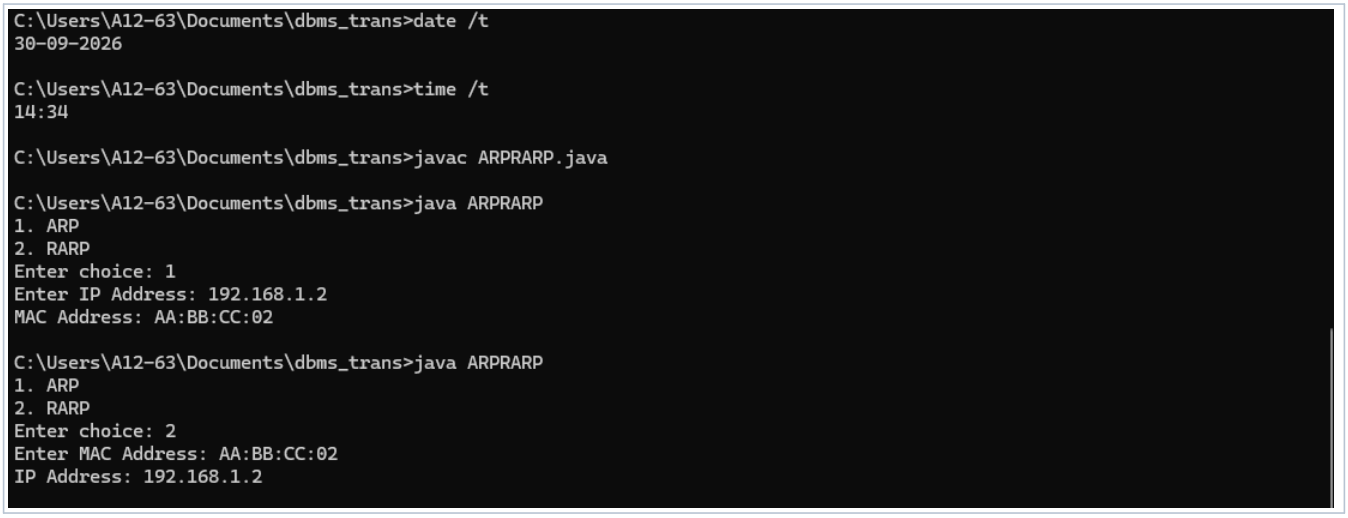
Running the script from Downloads prints the banner and local IP (192.168.6.72), then stops with 'Administrator permission is required'. Raw sockets on Windows need elevated rights, so close VS Code and reopen it with 'Run as administrator', then run the script again.



Part B: ARP and RARP program (Java)

Aim: To simulate the Address Resolution Protocol (ARP), which maps an IP address to a MAC address, and the Reverse ARP (RARP), which maps a MAC address back to an IP address.

Step 1: Compile and run
Compile with `javac ARPRARP.java` and run with `java ARPRARP`. A menu offers 1. ARP and 2. RARP.



Result: Choice 1 with IP 192.168.1.2 returns MAC address AA:BB:CC:02. Choice 2 with MAC AA:BB:CC:02 returns IP address 192.168.1.2, so the lookup works in both directions. (Run on 30-09-2026.)

Part C: Distance vector routing (Java)

Aim: To compute the shortest distance between every pair of routers from a cost matrix, as in distance vector routing (Bellman-Ford style updates).

Step 1: Compile and run
Compile with `javac distance.java` and run with `java distance`. Enter 3 routers and the cost matrix: row 1: 0 2 9, row 2: 2 0 3, row 3: 9 3 0.

```
C:\Users\A12-63\Documents\dbms_trans>date /t
30-09-2026

C:\Users\A12-63\Documents\dbms_trans>time /t
14:37

C:\Users\A12-63\Documents\dbms_trans>javac distance.java

C:\Users\A12-63\Documents\dbms_trans>java distance
Enter number of routers: 3
Enter cost matrix:
0 2 9
2 0 3
9 3 0

Shortest Distance Matrix:
0       2       5
2       0       3
5       3       0
```

Result: The shortest distance matrix is 0 2 5 / 2 0 3 / 5 3 0. The direct cost between router 1 and router 3 was 9, but going through router 2 costs 2 + 3 = 5, so the program correctly replaces 9 with 5.

Conclusion

The Python environment and Scapy were installed and tested, the sniffer program was prepared (raw sockets need administrator rights on Windows), and the ARP/RARP and distance vector programs produced the expected outputs.